Ambient AI Voice Scribe

Project Plan & Structure | Aligned to NHA EOI (Standalone Voice Scribe category)

1. Project Description

Problem: Doctors in public healthcare facilities spend a large part of each consultation typing notes into records instead of attending to the patient. This reduces patient interaction time and often leads to incomplete or inconsistent documentation.

Solution: An Ambient AI Voice Scribe. The doctor selects a patient, taps **Start**, and talks normally with the patient. The app listens through the phone microphone, converts the conversation to text (multilingual), saves it against a Patient ID / Session ID, and an AI summarizer turns it into a structured clinical note. The doctor reviews, edits and approves the note, which can be exported in an ABDM-compliant (FHIR) format.

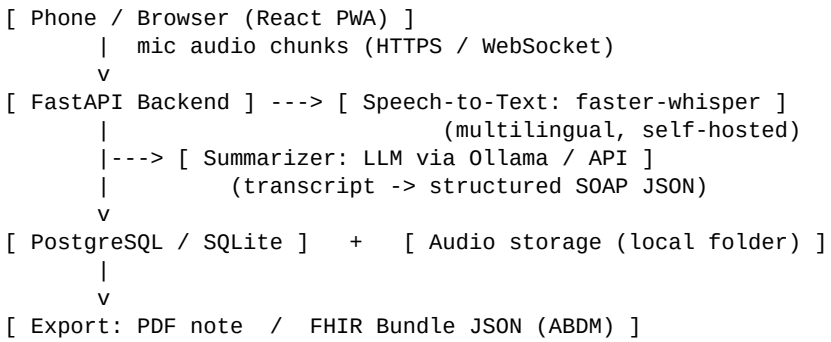
Key features: voice to text, multilingual support, structured output, editable notes, ABDM compliance, open-source tools only, suitable for deployment across public healthcare facilities.

Expected outcome: a working prototype that reduces documentation time and produces a clean, doctor-verified digital health record from a spoken consultation.

2. Workflow

Step	What happens
1	Doctor logs in, enters/selects **Patient ID**; system creates a **Session ID**.
2	Doctor taps **Start**; phone microphone records (browser MediaRecorder), audio sent in 10-30 s chunks.
3	Backend converts each chunk to text using the open-source speech model (voice-to-text API).
4	Transcript is **saved** in the database linked to Patient ID + Session ID.
5	Doctor taps **Stop**; full transcript is sent to the **AI summarizer** which returns structured JSON (SOAP note).
6	Doctor **edits** and approves the note; it is stored and can be exported as PDF / FHIR JSON.

3. System Architecture



Module descriptions

Module	Description
Frontend (React PWA)	The doctor-facing screens: login, Patient ID / Session ID entry, a Start/Stop recorder with live transcript, and an editable note form. Runs in any phone browser.
Audio capture	Records the microphone in short chunks so text appears while the consultation is still going on and long recordings are not lost.

Module	Description
Backend (FastAPI)	Central controller. Handles login, creates sessions, receives audio, calls the speech and summarizer services, and stores everything in the database.
Speech-to-text service	Open-source Whisper model that transcribes speech in English, Hindi, Bengali and other languages into text.
AI summarizer	Takes the full transcript and, using a fixed prompt, returns a structured note (complaint, symptoms, diagnosis, medicines, advice, follow-up) without adding facts that were not spoken.
Database	Stores doctors, patients, sessions, transcript chunks and notes, all linked by Patient ID and Session ID.
Export service	Converts the approved note into a printable PDF and an ABDM-compliant FHIR JSON document.

4. Tools & Technologies

Layer	Tool	Reason
Frontend	React + Vite (PWA), Tailwind CSS	Works on any phone browser with mic access; no app-store build needed
Audio capture	Browser MediaRecorder API	Native mic access, WebM/Opus chunks
Backend	Python FastAPI + Uvicorn	Fast, simple REST/WebSocket, easy ML integration
Voice-to-text	faster-whisper (Whisper medium/small); optional AI4Bharat / Bhashini for Indian languages	Open-source, multilingual (Hindi, Bengali, English etc.)
AI summarizer	Ollama with Llama 3 / Qwen (open-source); API model as fallback for prototype	Open-source requirement; structured JSON output
Database	SQLite (dev) -> PostgreSQL (prod), SQLAlchemy ORM	Simple start, scalable later
Auth	JWT (python-jose) + bcrypt	Doctor login, protects patient data
Interoperability	fhir.resources (Python)	Generates ABDM-compliant FHIR documents
PDF export	ReportLab	Printable consultation note
DevOps	Docker + docker-compose, Git/GitHub	One-command setup, version control
Testing	pytest, Postman	API and pipeline testing

5. Project Folder Structure

```
voice-scribe/
|-- backend/
|   |-- app/
|   |   |-- main.py                # FastAPI entry point
|   |   |-- config.py              # env settings, model names
|   |   |-- database.py            # DB connection / session
|   |   |-- models.py              # Doctor, Patient, Session, Transcript, Note
|   |   |-- schemas.py             # Pydantic request/response models
|   |   |-- auth.py                # login + JWT
|   |   |-- routers/
|   |   |   |-- patients.py        # create/search patient
|   |   |   |-- sessions.py        # start/stop session
```

```
|   |   |   |-- audio.py            # receive audio chunks
|   |   |   |-- notes.py            # summary, edit, approve, export
|   |   |-- services/
|   |   |   |-- stt_service.py      # faster-whisper transcription
|   |   |   |-- summarizer.py       # LLM prompt -> SOAP JSON
|   |   |   |-- fhir_service.py     # JSON -> FHIR Bundle
|   |   |   |-- pdf_service.py      # note -> PDF
|   |   |-- prompts/soap_prompt.txt # summarizer prompt template
|   |-- tests/                      # test_stt.py, test_summarizer.py, test_api.py
|   |-- requirements.txt
|   |-- Dockerfile
|-- frontend/
|   |-- src/
|   |   |-- main.jsx, App.jsx
|   |   |-- pages/
|   |   |   |-- Login.jsx
|   |   |   |-- PatientSelect.jsx   # Patient ID / Session ID entry
|   |   |   |-- Recorder.jsx        # Start/Stop + live transcript
|   |   |   |-- NoteEditor.jsx      # editable structured note
|   |   |-- components/
|   |   |   |-- RecordButton.jsx, TranscriptBox.jsx, NoteForm.jsx
|   |   |-- hooks/useRecorder.js    # MediaRecorder logic
|   |   |-- api/client.js           # axios calls to backend
|   |-- package.json, vite.config.js, Dockerfile
|-- data/audio/                     # saved recordings
|-- docs/                           # report, diagrams, demo screenshots
|-- docker-compose.yml
|-- .env.example
|-- README.md
```

6. Database Design

Table	Key fields
doctors	id, name, email, password_hash, hospital
patients	id (Patient ID), name, age, gender, ABHA_id (optional)
sessions	id (Session ID), patient_id, doctor_id, language, status, started_at, ended_at, audio_path
transcripts	id, session_id, chunk_no, text, timestamp
notes	id, session_id, summary_json, edited_json, status (draft/approved), fhir_json, updated_at

7. API Endpoints

Method	Endpoint	Purpose
POST	/auth/login	Doctor login, returns JWT
POST	/patients	Create or fetch patient by Patient ID
POST	/sessions	Start session -> returns Session ID
POST	/sessions/{id}/audio	Upload audio chunk -> returns transcribed text
POST	/sessions/{id}/stop	End session, trigger summarization
GET	/sessions/{id}/transcript	Full saved transcript
GET / PUT	/sessions/{id}/note	Fetch / edit structured note
POST	/sessions/{id}/approve	Approve note, generate FHIR
GET	/sessions/{id}/export?format=pdf|fhir	Download note

8. Structured Note Format (Summarizer Output)

```
{
  "chief_complaint": "",
  "history_of_present_illness": "",
  "symptoms": [],
  "examination": "",
  "diagnosis": "",
  "medications": [{"name":"","dose":"","frequency":"","duration":""}],
  "investigations": [],
  "advice": "",
  "follow_up": ""
}
```

The summarizer prompt instructs the model to translate non-English speech to English, extract only what was spoken (no invented data), and return valid JSON in the above schema. Fields the doctor can edit before approval.

9. Build Path: What to Do First and How to Check

Golden rule: build and test each layer on its own first, then connect it to the next. Do not move to the next step until the **Pass check** succeeds. Keep 5 mock consultation recordings (English, Hindi, Bengali) as a test set from the start.

#	Do (files)	How to check (pass condition)
0 Wk 1	**Setup:** install Python 3.11, Node 18+, Git, ffmpeg; create repo and the folder structure; create virtual env.	`python --version`, `node -v`, `ffmpeg -version` all print versions; repo pushed to GitHub.
1 Wk 1	**Backend skeleton:** `main.py`, `config.py` with a `/health` route.	Run `uvicorn app.main:app --reload`; open `localhost:8000/docs`; `/health` returns {"status":"ok"}.
2 Wk 1-2	**Database:** `database.py`, `models.py` (5 tables).	Start the app; `app.db` is created; open it in DB Browser for SQLite and see all 5 tables.
3 Wk 2	**Patient and Session APIs:** `schemas.py`, `routers/patients.py`, `routers/sessions.py`, `auth.py`.	In Swagger (`/docs`): login gives a token; POST /patients and POST /sessions return a Patient ID and Session ID; new rows appear in DB; wrong token gives 401.
4 Wk 3	**Speech-to-text alone (no app):** `services/stt_service.py` + small test script on a saved .wav/.webm file.	Script prints text for each of the 5 test recordings. Measure accuracy with `jiwer` (WER) against what you actually said; note WER per language and pick model size (small / medium).
5 Wk 3	**Audio upload endpoint:** `routers/audio.py` saves chunk, calls stt_service, stores text in transcripts.	`curl -F file=@test.webm .../sessions/1/audio` returns text; row added in transcripts with the right session_id.
6 Wk 4	**Frontend skeleton:** Vite + React, `Login.jsx`, `PatientSelect.jsx`, `api/client.js`.	`npm run dev` opens in browser; login works against the backend; entering Patient ID creates a session and shows Session ID on screen.
7 Wk 4	**Recorder (no backend yet):** `hooks/useRecorder.js`, `RecordButton.jsx`, `Recorder.jsx`.	Click Start: browser asks mic permission; console logs a chunk every ~10 s; Stop and play back the recorded blob to hear your voice. (Phone mic needs HTTPS or localhost; for phone testing use same Wi-Fi + an ngrok HTTPS link.)
8 Wk 5	**Connect live transcription:** send chunks to /audio, show text in `TranscriptBox.jsx`.	Speak a sentence; text appears on screen within a few seconds; refresh the page and the saved transcript loads from DB (GET /transcript).

#	Do (files)	How to check (pass condition)
9 Wk 6	**Summarizer alone:** `prompts/soap_prompt.txt`, `services/summarizer.py`; test with 5 typed sample transcripts.	Output is valid JSON matching the schema every time; nothing invented (compare with transcript line by line); medicines/doses are correct; Hindi/Bengali input gives English fields.
10 Wk 6	**Stop and summarize:** /stop calls summarizer, saves to notes table.	Full run: Start, talk 1-2 min, Stop; note appears in under 30 s; row exists in notes with status = draft.
11 Wk 7	**Edit and approve:** `NoteEditor.jsx`, `NoteForm.jsx`, `routers/notes.py`.	Edit a field, refresh: the edit persists; click Approve: status changes to approved and the note becomes read-only.
12 Wk 8	**Export:** `services/fhir_service.py`, `pdf_service.py`.	PDF opens with correct patient and note data; FHIR JSON loads without error in `fhir.resources` and passes the HL7 FHIR validator (R4).
13 Wk 9-10	**Docker, testing, docs:** `docker-compose.yml`, `tests/`, README, report.	`pytest` passes; `docker-compose up` on a clean machine runs the whole app; final run on all 5 recordings gives WER, latency and doctor rating for the report.

10. Evaluation Metrics

- Word Error Rate (WER) of transcription per language.
- Summary accuracy: doctor rating (1-5) and field-level correctness vs. manual note.
- End-to-end latency: Stop pressed -> note displayed (target under 30 s).
- Edit rate: percentage of note fields changed by the doctor.

11. Privacy & Compliance Checklist

- Patient consent screen before recording starts.
- HTTPS everywhere, JWT auth, encrypted password hashes, role-based access.
- Self-hosted models so audio never leaves the server (open-source requirement).
- FHIR R4 output mapped to ABDM profiles; optional ABHA ID linkage.
- Audit log of who viewed/edited each note.